

Dynamic TDMA for Wireless Sensor Networks: Principles, Critical Analysis and a Simulation Study

Vasilis Kladis

1092700

Department of Electrical and Computer Engineering, University of Patras

Abstract—Dynamic TDMA protocols for wireless sensor networks assign slots only to the nodes expected to transmit, which reduces idle listening. This report studies the cross-layer dynamic TDMA of Benrebbouh and Louail, which orders slots along the routing tree and predicts demand from past slot usage, either in the last frame (BSPS) or in all previous frames (BAPS). After reviewing the background, we analyze the protocol and compare it in simulation with its random baselines, a static tree-ordered schedule, other predictors and an oracle, with duty-cycled CSMA as a reference outside TDMA. The reported latency advantage over random schedules results from the tree ordering, which a static schedule also provides without runtime computation. Because a node without a slot cannot announce new data, an idle node retains a slot in every second frame, and up to half of the readings are delayed by one sampling period. BAPS can serve an event-driven node in only half of the frames indefinitely, and a realistic distribution of the rebuilt schedule costs more energy than prediction saves. Prediction yields lower latency than the static schedule only when node activity persists without interruption. Decaying weights in BAPS, as we propose, bound its reaction times; it improves on BSPS only if the frames without a slot are also ignored, and only when active nodes report intermittently. CSMA achieves lower latency under light load, but at equal channel time it consumes at least twice the energy of TDMA, and part of its readings are lost.

Index Terms—Wireless sensor networks, TDMA, dynamic slot assignment, cross-layer design, convergecast, energy efficiency.

I. INTRODUCTION

WIRELESS sensor networks (WSNs) consist of small battery-powered devices, called nodes, that measure a quantity such as temperature or humidity and report it to a collection point, the sink [1]. A network monitoring a vineyard, a forest or an industrial plant may contain hundreds of nodes. Fig. 1(a) shows the seven-node example used throughout this report.

The design of such networks is subject to four restrictions:

- 1) *Energy*. Batteries can rarely be replaced in practice, so their capacity determines the lifetime of the network.
- 2) *Range*. Low-power radios have a range of several tens of meters, so most data are relayed hop by hop to the sink along a *routing tree* (arrows in Fig. 1(a)).

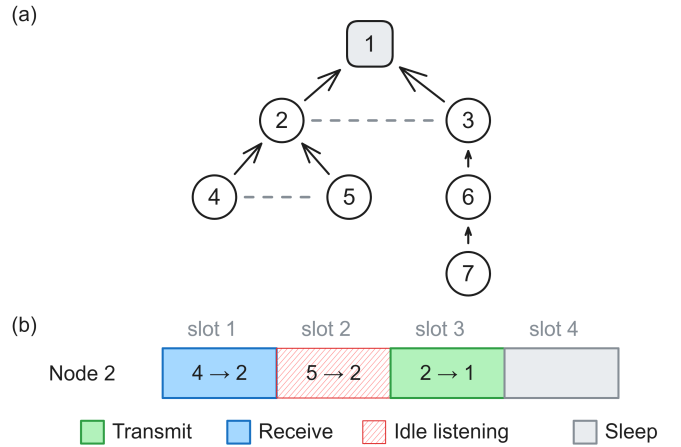


Fig. 1. Example network and idle listening under static TDMA. (a) Node 1 is the sink, arrows form the routing tree along which data travel, and dashed lines are radio links that carry no data but still cause interference. (b) The radio of node 2 during one frame of a static TDMA schedule; node 2 must listen during the slot of node 5, which has no data to transmit.

- 3) *One shared channel*. All nodes share one frequency, so nearby simultaneous transmissions interfere and are lost.
- 4) *Half-duplex radios*. A node cannot transmit and receive simultaneously.

The radio accounts for most of the energy consumption, and listening to an idle channel requires nearly as much energy as reception. On IEEE 802.15.4 hardware, a slot of idle listening draws approximately ten times the charge of a sleep slot [2]. The *medium access control* (MAC) protocol, which decides when each radio transmits, listens or sleeps, therefore largely determines the network lifetime [3]. It must avoid four sources of waste [4]: *collisions*, which destroy packets that must then be retransmitted; *overhearing* of packets addressed to other nodes; *control overhead* spent on coordination instead of data; and *idle listening* for traffic that does not arrive.

Time division multiple access (TDMA) removes the first two. Time is divided into *slots*, grouped into repeating *frames*, and each node transmits only in its own slots, so transmissions never collide and a node can put its radio to

sleep in the slots of other nodes [3], [4]. A fixed, or *static*, schedule, however, assigns every node a slot in every frame, even when the node has no data to transmit. The receiver of the node must still listen during that slot, as node 2 does for node 5 in Fig. 1(b), and the unused slot lengthens the frame, which delays every packet.

Dynamic TDMA rebuilds the schedule so that only nodes with data to transmit are assigned slots [5]. The protocol studied here, by Benrebbouh and Louail [1], does so at the sink in every frame. Because a node without a slot cannot announce new data, the sink predicts which nodes will need one. It grants a slot to the nodes that used their slot in the previous frame (the predictor BPS, based on a single previous state) or in at least half of all previous frames (BAPS, based on all previous states), and again to every node denied in the previous frame (the *re-probe* rule), so that no node is excluded for two consecutive frames. It also uses the routing tree to place each node's slot after those of its children, so that data can be delivered to the sink within one frame. The authors report lower latency, energy consumption and frame length than with random schedules [1].

This report examines four problems that the original evaluation does not address:

- 1) *Attribution*. The protocol differs from its baselines in three respects (it adapts to traffic, orders slots along the tree and uses a less restrictive interference rule), so it is unclear which change produces the reported gains.
- 2) *Re-probing*. The re-probe rule includes an idle node in the schedule of every second frame, which limits the slots that can be removed and delays readings.
- 3) *Memory*. A prediction based on all previous frames responds more slowly as the operating time of the network increases, and can serve a node that becomes active in only every second frame.
- 4) *Update cost*. Rebuilding and distributing a new schedule at every frame consumes energy that the original evaluation does not include.

We adopt the assumptions of the studied protocol. Data flow only toward the sink and are merged at every hop, all nodes share one perfectly synchronized channel, interference depends only on radio range, and packets are lost only in collisions. As a reference outside TDMA, we also simulate a simple contention-based protocol with a listen/sleep cycle (duty-cycled CSMA, Section II-D).

Besides reproducing the published comparison, the report makes three contributions. It separates the effects of ordering, interference rule and adaptation under two time models. It derives four propositions on the re-probe rule and the memory of the predictors, and proposes a version of BAPS with decaying weights (D-BAPS) and a variant that counts only the frames in which a node holds a slot (EWMA). It determines the conditions under which prediction is beneficial, for bursty and intermittent traffic and sampling periods of 30 to 400 slots.

Section II describes the energy consumption of a sensor node and its reduction by MAC protocols. Section III

TABLE I
MEASURED CHARGE PER 15 MS SLOT OF AN IEEE 802.15.4 NODE [2] AND ITS RATIO TO THE CHARGE OF A SLEEP SLOT

Radio state	Slot type in [2]	Charge (μC)	$\times$ Sleep
Transmit	TxData	69.6	14.2
Receive	RxData	72.1	14.7
Idle listening	Idle	47.9	9.8
Sleep	Sleep	4.9	1.0
Transmit + ACK	TxDataRxAck	92.6	18.9
Receive + ACK	RxDataTxAck	96.3	19.7

formalizes TDMA scheduling, and Section IV the design choices that distinguish TDMA schedulers. Section V presents the studied protocol, and Section VI analyzes the four problems and proposes D-BAPS and EWMA. Sections VII and VIII describe and report the simulation study, Section IX discusses the results, and Section X concludes.

II. ENERGY AND MEDIUM ACCESS IN WIRELESS SENSOR NETWORKS

A. Nodes, Sink and Traffic

A node combines a sensor, a microcontroller, a low-power radio and a battery; the microcontroller and the radio consume most of its energy [2]. Because radio range is short, data are relayed hop by hop and converge on the sink. This many-to-one pattern, called *convergecast*, is the most common; of the 76 schedulers for time-slotted channel hopping (TSCH), the TDMA mode of IEEE 802.15.4, surveyed in [6], 70 target it. Convergecast also concentrates traffic near the sink, whose neighbors relay the data of entire subtrees [6].

Traffic depends on the application and is commonly classified as periodic, aperiodic or on-demand [5]. Under periodic traffic every node generates a fixed number of packets per frame, and a fixed schedule is sufficient. When nodes generate data at different rates, or only when an event occurs, “using the same TDMA-schedule for a long duration may result in the wastage of bandwidth” [5], and dynamic scheduling becomes useful.

B. Radio Energy Consumption

At any moment a node's radio transmits, receives, listens for a packet that has not arrived, or sleeps. Table I gives the charge drawn in each state during one 15 ms slot on an IEEE 802.15.4 node [2].

Idle listening consumes almost as much charge as reception. The transceiver draws 11.6 mA in both states, and a receiver cannot detect that no packet will arrive until it has listened, for a guard time of 2.6 ms in the measured implementation [2]. A sleep slot draws approximately one tenth of the charge of an idle-listening slot.

A MAC protocol saves energy mainly by lowering the *duty cycle*, the fraction of time during which the radio is active. A node that draws a charge Q_f (in coulombs) per

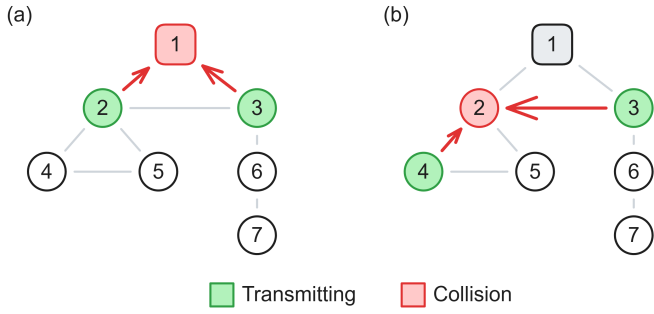


Fig. 2. Collisions on the example network. (a) Direct collision at the sink. (b) Hidden-terminal collision at node 2; nodes 3 and 4 are outside each other's range, so carrier sensing cannot prevent it. Gray lines are radio links.

frame of N_f slots of duration T_s (in seconds) from a battery of capacity B (in mAh) lasts [2]

$$T_{\text{life}} = \frac{3.6 B}{Q_f} \cdot \frac{N_f T_s}{86400} \text{ days.} \quad (1)$$

Every slot spent listening instead of sleeping adds $43 \mu\text{C}$ to Q_f .

C. Collisions and the Hidden-Terminal Problem

Two transmissions that overlap in time at a receiver collide; neither can be decoded, and both must be retransmitted [4]. There are two cases [1] (Fig. 2). In a *direct collision* the transmitters are within range of each other, as are nodes 2 and 3, whose packets collide at the sink. In a *hidden-terminal* collision they are not; nodes 3 and 4 are outside each other's range, yet both are within range of node 2. Listening before transmitting (carrier sensing) reduces the first kind but not the second, since neither hidden node can detect the transmission of the other.

In both cases the conflict occurs at the receiver, so two transmissions can share a slot only if neither transmitter is within range of the other's receiver. The scheduling rules of Section III are based on this condition.

D. Contention-Based and Schedule-Based Access

MAC protocols for WSNs fall into two families [4]. In *contention-based* protocols, which, like Wi-Fi, use carrier-sense multiple access (CSMA), a node with a packet listens to the channel, transmits if it is free and otherwise backs off for a random time. No coordination is needed, and at low load packets are sent without delay, but collisions increase with load, the hidden-terminal problem persists, and the radio of a node must remain on to receive traffic addressed to it [4]. S-MAC reduces this idle listening with a common sleep and wake-up schedule for neighboring nodes, at the cost of higher latency [4].

In *schedule-based* protocols, mainly TDMA, each node owns the slots in which it may transmit, so collisions are excluded by construction, and a node sleeps in the slots that are neither its own nor its children's, which also removes overhearing [3], [4]. The costs are time synchronization,

with a guard time in every slot (2.6 ms of a 15 ms slot in [2]), the computation and distribution of the schedule [5], and, for a fixed schedule, no response to changes in demand. *Hybrid* protocols combine the two. In Z-MAC, the owner of a slot has priority instead of exclusive use, and other nodes may contend for a slot that its owner does not use; on a 42-node testbed, scheduling was advantageous only above a certain load [7].

E. Idle Listening in Static TDMA

A static schedule assigns every node a slot in every frame. A node without data can sleep during its own slot (an unused transmit slot costs the same as sleep in the model of [2]), but its parent cannot determine in advance that the slot will remain empty and must listen (Fig. 1(b)). The unused slot thus costs the receiver $47.9 \mu\text{C}$ of idle listening instead of $4.9 \mu\text{C}$, and it lengthens the frame, which delays every packet. This waste increases as the traffic becomes sparser. Removing it requires information, before each frame, on which nodes will have data to transmit, yet a node without a slot cannot signal that it now requires one. Sections IV and V describe methods by which a scheduler can obtain this information.

III. A FORMAL MODEL OF TDMA SCHEDULING

A. Network Model and Routing Tree

The network is a graph $G = (V, E)$ whose vertices $V = \{v_1, \dots, v_N\}$ are the nodes, v_1 being the sink, and whose edges join nodes within communication range r of each other, the unit-disk graph (UDG) model of [1], [8]. For N nodes distributed uniformly over a square of side a , the density

$$\sigma = \frac{\pi r^2 N}{a^2} \quad (2)$$

is the average number of neighbors of a node [1], [8].

The routing tree T assigns every node except the sink one parent, to which it forwards its data, and is stored as a parent vector $P = [P_i]$, P_i being the parent of v_i [8]. Since [1] does not specify the construction of its tree, we use the standard minimum-hop tree [3], [9]; the sink broadcasts a beacon with depth 0, and each node adopts a neighbor with the smallest advertised depth as parent and rebroadcasts that depth plus one.

We write $P(v)$, $C(v)$ and $d(v)$ for the parent, children and depth (hops to the sink) of node v ; nodes without children are leaves. In the example (Fig. 1(a)), $P = [-, 1, 1, 2, 2, 3, 6]$; nodes 2 and 3 are at depth 1, nodes 4–6 at depth 2 and node 7 at depth 3. The links 2–3 and 4–5 are not in the tree but still constrain the schedule (Section III-C).

B. Frames and the Schedule Matrix

Each slot carries one packet, and a frame of L slots repeats indefinitely; L is the *frame length*. A schedule specifies for every node and slot whether the node transmits, receives or sleeps. Following [1], [8], we write it as an $N \times L$ matrix $Slots$ whose entry in row i and column j is i if v_i

TABLE II
SCHEDULE MATRIX OF THE RUNNING EXAMPLE ($L = 4$)

	Slot 1	Slot 2	Slot 3	Slot 4
v_1 (sink)	0	0	2	3
v_2	4	5	2	0
v_3	0	6	0	3
v_4	4	0	0	0
v_5	0	5	0	0
v_6	7	6	0	0
v_7	7	0	0	0

Bold entries denote transmissions, other nonzero entries reception from the indicated child, and 0 sleep.

transmits in slot j , k if v_i receives from its child v_k , and 0 if v_i sleeps.

Table II gives a schedule for the example, built in Section III-D. One slot may carry several transmissions; in slot 1, nodes 7 and 4 transmit to receivers that are outside the range of each other's sender. This *spatial reuse* shortens the frame. The matrix also determines the radio state of every node in every slot, and therefore, through Table I, its duty cycle and energy.

As in [1], each node transmits once per frame, merging its children's data with its own into one packet (*aggregated convergecast*).

C. Interference Model and Conflict Rules

The condition of Section II-C depends only on radio range and is called the *protocol model* of interference. Ergen and Varaiya distinguish two kinds of conflict [3]. A *primary conflict* arises when a node would have to transmit and receive in the same slot, or receive two packets in the same slot, which a single half-duplex transceiver cannot do. A *secondary conflict* arises when a receiver is also within range of another transmitter; these are the collisions of Section II-C. Scheduling rules differ in which nodes they consider receivers; two rules are in use.

Rule A (two-hop exclusion). Two nodes within two hops of each other never share a slot. This is the rule stated in [1] ("nodes that are one-hop or two-hop neighbors do not have the same communication slot") and the hop-count interference model of [5] with $k = 2$. It treats every neighbor of a transmitter as a potential receiver, as a broadcast requires. In graph terms it is a *distance-2 vertex coloring*, in which nodes up to two hops apart receive different colors (slots).

Rule B (parent-aware exclusion). In convergecast, however, each transmission has a single intended receiver, the transmitter's parent, and Bouchédjé and Louail use this property to relax Rule A [8]. Two nodes conflict if

- 1) they are one-hop neighbors in G , or
- 2) they are two-hop neighbors connected by an intermediate node that "is the parent of at least one of these two nodes".

A collision at a node that is the parent of neither transmitter has no effect, since neither packet is addressed to

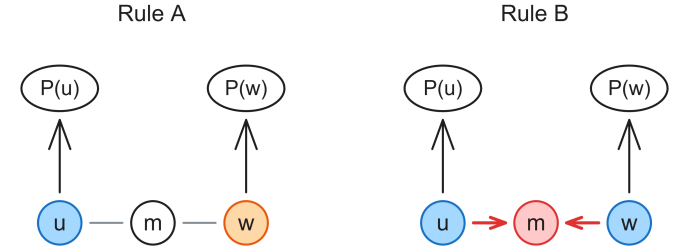


Fig. 3. The configuration in which Rules A and B differ. Nodes u and w are two hops apart through m , which is the parent of neither; colors denote slots, and arrows point from child to parent. Rule A assigns u and w different slots. Rule B allows them to share one; their transmissions then collide at m (red), which is the intended receiver of neither.

it. Fig. 3 shows the smallest case in which the rules differ; nodes u and w are two hops apart through m , the parent of neither, so Rule A assigns them different slots and Rule B allows them to share one.

Rule B requires the routing tree (Section IV). Both rules exclude primary conflicts, since a parent and its child are neighbors and two children of one parent are two hops apart through it. The studied protocol states Rule A but describes its own relaxation as sharing slots among "nodes that are not one-hop or two-hop neighbors in the routing tree, even if they are neighbors in the network" [1]. Read literally, this rule measures distance in the tree instead of the network; we call this reading the *tree-distance criterion*. It would allow nodes 3 and 4 (three hops apart in the tree) to share a slot, recreating the hidden-terminal collision of Fig. 2(b). We therefore read the protocol as using Rule B of the earlier scheduler of Bouchédjé and Louail [8]. In the example the two rules yield the same schedule; nodes 4 and 7, which share slot 1, are four hops apart, and nodes 5 and 6, which share slot 2, are three. Section VIII quantifies their difference in denser networks.

D. Greedy Coloring and a Lower Bound

Finding the shortest conflict-free schedule is NP-complete [3], [9], so practical schedulers, including the studied one, are heuristics, mostly variants of *greedy coloring*. The nodes are processed in a given order, and each is assigned the first slot that does not conflict with those already assigned; a new slot is added only when no existing slot is feasible, so the result is always valid.

Greedy schedulers differ mainly in the order in which they process the nodes. The random static baseline of [1] uses a random order, the routing-aware schedulers of [8] process the deepest nodes first, preferring the node that forwards the most packets, and the protocol of [1] processes the tree from the leaves toward the sink. The routing-aware schedulers also refine the search for a slot; a leaf searches from the first slot, whereas any other node starts "from the highest slot of its children" and continues circularly, adding a new slot only if no existing slot is feasible [8].

Processing the deepest node first, this procedure produces Table II. Node 7 is assigned slot 1. Node 6 cannot

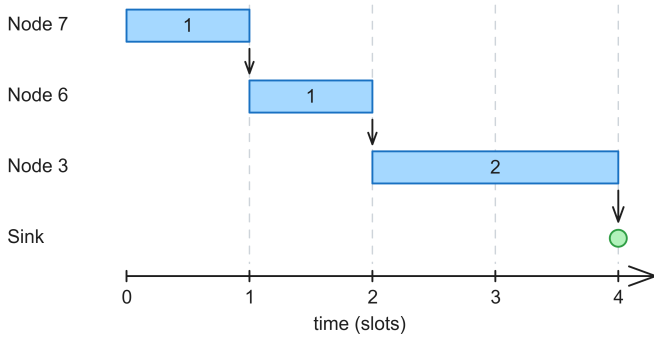


Fig. 4. Latency as a sequence of waiting times. Each bar gives the number of slots that the packet of node 7 waits at a node on its path for that node's slot, and each arrow a transmission; under Table II the packet is delivered to the sink after $1 + 1 + 2 = 4$ slots.

transmit in slot 1, in which it receives from node 7, so slot 2 is added for it. Node 4 shares slot 1 with node 7, since the two are neither neighbors nor two hops apart, and node 5, a neighbor of node 4, shares slot 2 with node 6. Node 2 receives in slots 1 and 2 and is assigned a new slot 3. No existing slot is feasible for node 3; slot 3 carries another transmission to the sink, slot 1 would cause a collision with node 4 at node 2, and in slot 2 node 3 receives from node 6, so a fourth slot is added.

Heuristic schedules can be compared with a lower bound. Even without interference, an aggregated frame requires at least $\Delta(T)$ slots, the largest number of tree links meeting at one node, since a node can use only one of its links per slot [9]. In the example $\Delta(T) = 3$, at node 2. The transmissions $4 \rightarrow 2$, $5 \rightarrow 2$, $2 \rightarrow 1$ and $3 \rightarrow 1$ conflict pairwise, $3 \rightarrow 1$ with the first two through the link 2–3, so the four slots of Table II are optimal. Section VIII uses $\Delta(T)$ as a reference.

E. Latency

At every node on its path, a packet waits in the queue until that node's slot and is then forwarded to the parent; its latency is the sum of these waiting times. Fig. 4 traces the packet of node 7 under Table II. It is transmitted in slot 1, waits one slot at node 6 (until slot 2) and two at node 3 (until slot 4), and arrives at the sink after $1 + 1 + 2 = 4$ slots.

Each waiting time depends only on the slots of a node and its parent. If the parent's slot is later in the frame, the packet waits for the difference; if earlier, the packet waits for the parent's slot in the next frame. With s_i the slot in which node i transmits and time counted from the start of the frame in which the packet is generated, the latency of the packet of node v_i along the path $i = n_0 \rightarrow n_1 \rightarrow \dots \rightarrow n_k \rightarrow v_1$ is [8]

$$dt_i = s_{n_0} + \sum_{h=0}^{k-1} nbrSlots(n_h, n_{h+1}), \quad (3)$$

where the waiting time between a node i and the next node j on the path is

$$nbrSlots(i, j) = \begin{cases} s_j - s_i, & s_i < s_j, \\ (L - s_i + s_j) \bmod L, & s_i > s_j. \end{cases} \quad (4)$$

The first term of (3) is the waiting time for the node's own slot, and the average latency is the mean of dt_i over all nodes except the sink [8]. The second case of (4) gives the longest waiting times, up to $L - 1$ slots. In Table II every node transmits before its parent, so every packet is delivered to the sink within the frame in which it is generated; the latency is 3 slots for nodes 2, 4 and 5 and 4 slots for nodes 3, 6 and 7, or 3.5 slots on average. Section IV-C examines the effect of reversing this order.

IV. CLASSIFICATION OF TDMA SCHEDULERS

TDMA schedulers differ in the location of the schedule computation, the lifetime of the schedule, the use of the routing tree, and the estimation of demand. The first two axes form the classification used in [1] after [5]. The third, we argue, explains the latency gain reported in [1], and the fourth, implicit in the literature, determines the performance that a dynamic scheduler can achieve. Table IV places the schemes discussed in this report along the four axes.

A. Centralized and Distributed Scheduling

In *centralized* scheduling one node, usually the sink, collects the topology, computes the complete schedule and distributes it to the nodes. The complete view permits optimal or near-optimal schedules, but the approach requires global synchronization and is suitable “neither for the large networks nor for the networks in which the network topology changes frequently” [5]. In *distributed* scheduling each node chooses its own slots by exchanging messages with its neighbors, which scales to large networks, adapts to topology changes and requires only local synchronization but rarely attains the optimum [5].

The studied protocol is centralized; the sink collects the topology, the routing tree and the slot usage of every node, and rebuilds and distributes the schedule at every frame [1]. Its authors later proposed a distributed variant [10].

B. Static and Dynamic Scheduling

“In static TDMA-scheduling, once a schedule is generated, the same schedule is used for sufficiently long time, without performing the rescheduling”, whereas in dynamic scheduling slots are reassigned per slot or per frame, so frequently that computing and distributing the schedule must have a low cost [5]. Rescheduling alone does not make a scheduler responsive to traffic; the random dynamic baseline of [1] draws a new random order at every frame but still assigns every node a slot. We call a scheduler *traffic-adaptive* when the set of nodes that are assigned

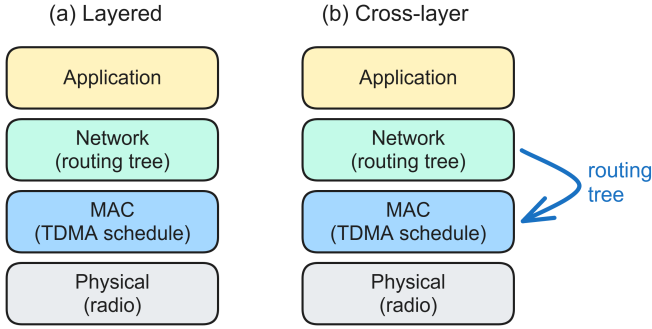


Fig. 5. Protocol layers of a sensor node. (a) In a layered design the MAC layer uses only the connectivity of the network. (b) In a cross-layer design the TDMA scheduler also reads the routing tree held by the network layer.

slots depends on their demand; only such schedulers can shorten the frame when nodes are idle.

Two routing-aware schedulers from the same research group illustrate the difference. Efficient-Depth-ReLO (Bouchédjara and Louail, 2020 [8]) is static; it computes one schedule, such as Table II, and repeats it in every frame. BSPS (Benrebouh and Louail, 2021 [1]) rebuilds the schedule at every frame; in the example of Section V-D, when nodes 5, 6 and 7 have been idle, it excludes them, and the frame is shortened from four slots to three.

C. Routing-Aware Scheduling and Cross-Layer Design

A node's communication software is organized in layers, each with its own task and information (Fig. 5). The application layer produces the readings, the network layer determines the route to the sink and holds the node's parent in the routing tree, the MAC layer decides when the radio transmits, receives or sleeps and holds the schedule, and the physical layer is the radio itself. In a strictly layered design each layer uses only the services of adjacent layers and never reads their internal state, so that one layer, such as the routing protocol, can be replaced without changing the others. A layered scheduler therefore uses the connectivity of the network but not the routes of the data.

A *cross-layer* design deliberately allows one layer to read the state of another. In routing-aware TDMA the scheduler reads the parent vector P , which identifies the single receiver of every transmission, permitting Rule B (Section III-C), and provides the depth of every node, permitting children to be scheduled before their parents. Cross-layering also couples the schedule to the tree, so the schedule must be rebuilt whenever a route changes. Table III compares the two designs.

Ordering has the largest effect on latency. Consider another valid schedule for the example with the same frame length and the same six transmissions in a different order, with node 3 in slot 1, nodes 4 and 6 in slot 2, node 2 in slot 3, and nodes 5 and 7 in slot 4. The packet of node 7 is now transmitted in slot 4, after the slot of node 6, waits for that slot in the next frame, and then waits again for the slot of node 3. By (3)–(4), $dt_7 = 4 + 2 + 3 = 9$ slots, compared

TABLE III
PROPERTIES OF A TDMA SCHEDULER IN A LAYERED AND A CROSS-LAYER DESIGN

	Layered	Cross-layer
Input	Connectivity	Connectivity and the routing tree (parents, children, depths)
Slot order	Arbitrary	Children before parents
Conflict rule	Rule A	Rule B
Relay nodes	Not identified	Identified; active while their children transmit
Route change	Schedule unaffected	Schedule recomputed
Modularity	Layers replaceable independently	MAC coupled to the routing protocol

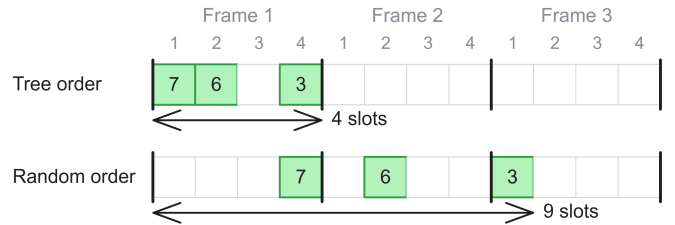


Fig. 6. Effect of slot ordering on latency. Green cells mark the slots in which nodes 7, 6 and 3 forward the packet of node 7. When every node transmits after its children (Table II), the packet traverses the three hops within one frame; under the reordered schedule described in the text, it waits for a new frame at every hop.

with 4 under Table II (Fig. 6), and the average latency rises from 3.5 to 4.67 slots. When children are scheduled before their parents, every packet is delivered to the sink within the frame in which it is generated; otherwise a packet can wait up to $L - 1$ additional slots at every hop. Ordering is a static property of the schedule, independent of traffic.

Among static routing-aware schedulers, Bouchédjara and Louail found that CoLaNet, which colors the tree starting from the node with the most neighbors, produces the shortest frames, yet a frame of minimal length “does not guarantee minimal latency” [8], as Section VI-A also shows for the studied protocol. The ordering constraint can be satisfied by a node with a global view (the sink, in [1]) or by a distributed procedure that proceeds from the leaves to the sink.

D. Estimation of Demand

A traffic-adaptive scheduler must determine before a frame begins which nodes will need slots in it. It can obtain this information in three ways.

Prediction. The scheduler infers demand from past behavior without signaling, as in [1], whose predictors check whether each node used its slot in past frames, and its extension [11]. Prediction requires no messages but can be incorrect, and a node wrongly denied a slot cannot correct the scheduler (Section II-E).

Request. Nodes request slots explicitly. In 6TiSCH, the IETF standard for IPv6 over TSCH, a node negotiates cells

(slots) with a neighbor through the 6P protocol; signaled requirements are more accurate than those inferred locally, at the cost of control messages [6].

Election. In TRAMA, a distributed TDMA protocol, the owner of each slot is elected among two-hop neighbors by pseudo-random priorities, and an owner with insufficient data releases its slots to other nodes [12]. Because each node decides on the basis of its own queue, TRAMA does not mispredict demand, but its priorities ignore the routing tree, so readings can incur the additional waiting times of Fig. 6. Hybrid protocols such as Z-MAC (Section II-D) need not estimate demand, since a slot left unused by its owner is used by other nodes through contention [7].

Prediction and request can be compared on the example. If node 5 did not use its slot in frame $f - 1$ and generates a reading at the start of frame f , BPS denies it a slot, and the reading waits until the re-probe in frame $f + 1$ (Section V-B) without any message. Under 6TiSCH the reading waits instead for a 6P exchange with the parent, which costs at least two control messages.

V. CROSS-LAYER DYNAMIC TDMA

A. Operating Principle

The protocol is a centralized, contention-free TDMA that uses the routing tree and is recomputed by the sink at every frame. It aims to avoid “waste in terms of unused slots by removing slots from nodes that have no data to transmit” [1], shortening the frame while retaining the routing-aware ordering of Section IV-C. Because a node denied a slot cannot announce pending data (Section II-E), the sink decides on the basis of each node’s *state*, a binary record of whether the node used the slot it was granted in a previous frame.

B. Frame Construction

Algorithm 1 builds one frame. The sink processes the routing tree from the leaves toward the root, handling the current leaves, removing them and repeating the procedure for the leaves of the remaining tree. A node is assigned a slot if at least one of three conditions holds:

- 1) it was denied a slot in the previous frame (the set D in Algorithm 1), which guarantees that no node is excluded for two consecutive frames; we call this the *re-probe* rule;
- 2) the predictor $pred(v)$ classifies it as active;
- 3) at least one of its children holds a slot in the frame being built, so it must relay; this *relay protection* maintains the connectivity of the tree whenever traffic is present in the subtree of a node.

The slot is chosen by the routine `ATTRIBUTESLOT`, which is invoked but not defined in [1]; we adopt it from the earlier scheduler of [8] (Section III-D). A leaf searches from the first slot, any other node from the slot after the highest slot of its children, and the frame is extended only when no slot is feasible. Unlike [8], we do not allow the search to wrap around to the first slots (Section VII-C). Since

Algorithm 1 Construction of frame f in [1]

Require: routing tree T ; set D of nodes denied a slot in frame $f - 1$; predictor $pred$
Ensure: schedule of frame f ; updated set D

```

1:  $R \leftarrow T$ 
2: while  $R$  contains nodes other than the sink do
3:   for all leaves  $v$  of  $R$  do
4:     if  $v \in D$  or  $pred(v)$  or a child of  $v$  holds a slot
       in frame  $f$  then
5:       ATTRIBUTESLOT( $v$ );  $D \leftarrow D \setminus \{v\}$ 
6:     else
7:        $D \leftarrow D \cup \{v\}$ 
8:     end if
9:   end for
10:  remove the leaves of  $R$  from  $R$ 
11: end while
```

children are processed before their parent, condition 3 can be evaluated when the parent is reached, and every child transmits before its parent. The frame length is the number of slots in use once all nodes have been processed, so it decreases whenever nodes are denied slots.

C. BPS and BAPS Predictors

Let $u_v(j) \in \{0, 1\}$ record whether node v used its slot in frame j ; a node that held no slot in frame j has $u_v(j) = 0$. The two variants of the protocol differ only in the predictor.

BSPS (based on a single previous state) grants a slot to a node that used its slot in the previous frame, that is, $pred(v) = u_v(f - 1)$. “If the node did not use its slot in the previous frame, it will be deprived of its slot in the current frame” [1].

BAPS (based on all previous states) grants a slot to a node that used its slot “at least in half of the previous TDMA frames” [1]:

$$pred(v) = \left\lceil \sum_{j=1}^{f-1} u_v(j) \geq \frac{f-1}{2} \right\rceil. \quad (5)$$

In the example of [1], a node obtains a slot in the seventh frame if it used its slot at least three times in the previous six. BAPS is preceded by a static TDMA of at least three frames, so that the sums are meaningful when it starts [1].

D. Worked Example

Suppose that the example network operates under the schedule of Table II in frame $f - 1$, and nodes 5, 6 and 7 have no data to transmit in it. BSPS builds frame f as follows. Among the leaves, node 4 used its slot and retains one, whereas nodes 5 and 7 did not and are denied; both are added to D . At the next level, node 2 retains its slot, which it used and requires for relaying the data of node 4, while node 6 is denied, since it did not use its slot and its only child, node 7, holds none. Node 3 used its slot and retains it. The frame is shortened to three slots, with node 4 in slot 1, node 2 in slot 2 and node 3 in slot 3. In

TABLE IV
POSITIONING OF TDMA SCHEDULERS ALONG THE FOUR AXES OF SECTION IV

Scheme (year)	Computed by	Schedule lifetime	Use of the routing tree	Demand estimated by
Random static TDMA, baseline of [1]	Sink	Static	None	Not estimated (every node, every frame)
Random dynamic TDMA, baseline of [1]	Sink	Recomputed every frame	None	Not estimated (every node, every frame)
Efficient-Depth-ReLO (2020) [8]	Sink	Static	Ordering; Rule B	Not estimated (every node, every frame)
BSPS, BAPS (2021) [1]	Sink	Dynamic	Ordering; Rule B; relay protection	Prediction from slot usage
Level thresholds, LT (2023) [11]	Sink	Dynamic	Level-dependent thresholds	Prediction; node state
TRAMA (2003) [12]	Every node	Dynamic	None	Election; release announcements
TSCH with 6P (2012 onward) [6]	Sink or nodes	Dynamic	Scheduler-dependent	Explicit request, or local queue

frame $f + 1$, however, nodes 5, 6 and 7 belong to D and are assigned slots whether or not they have data to transmit, and the frame length returns to four slots. Section VI shows that this alternation is general.

E. Design Heuristics

The authors list five techniques by which the protocol reduces energy consumption and latency [1]. Nodes that are not within two hops of each other in the routing tree share slots, “even if they are neighbors in the network” (Section III-C); a parent’s slot is placed “as close as possible” after those of its children; the deepest nodes are prioritized, since they “accumulate the latencies of the nodes on their way to the sink”; the frame length changes “according to the number of nodes that have data to transmit”; and switching between radio modes is reduced, “such that each node switches from the sleep to the active state only when needed”. Only the fourth is specific to dynamic scheduling; the others are properties of the slot assignment or of TDMA in general and apply equally to a static schedule.

F. Original Evaluation

The protocol was evaluated with the JUNG framework on 100 networks of 20 to 200 nodes, deployed uniformly in a square of side $a = 540$ m with a communication range of 50 m under the UDG model, the sink at the center and densities $\sigma \in [2, 20]$; each scheduler ran for 50 frames [1]. The reported metrics are the average latency, the frame length, the average energy consumption per node and the duty cycle, defined as “the ratio of the period where a node were active to the total TDMA length” [1]. Both variants are reported to outperform the random static and random dynamic baselines on all four. The results are shown graphically and discussed qualitatively, without numerical values in the text.

G. Extension with Level-Dependent Thresholds

Chittapragada et al. proposed three modifications [11]. First, the fixed threshold of BAPS becomes one that depends on the node’s level (depth) in the tree, decreasing

linearly from $K_{\text{leaf}} = 60\%$ at the deepest level to $K_{\text{top}} = 30\%$ at depth 1, so that relays retain their slots at a lower usage ratio, which is computed over the slots allocated to the node. We call this scheme LT (level thresholds). Second, a *PriorityScore* combines four measured values of each node (battery level, queue length, data rate and channel quality), and third, slot lengths are made proportional to it. Nodes denied a slot “are provided the slots in the next cycle”, as in [1]. The evaluation compares the number of active nodes against BSPS and BAPS for networks of 10, 100 and 1000 nodes and concludes that the proposal “performed similarly to, and even slightly better than, the original algorithm” [11].

VI. CRITICAL ANALYSIS

We derive each of the four problems of Section I from the mechanism of Section V and verify it with the simulator of Section VII. Every *round* of 100 slots (1.5 s) starts with one TDMA frame, after which the radios sleep, so the *sampling period* is one round. At the start of a round each node generates a reading with probability p , the *offered load*, independently of the past (*memoryless* traffic); a reading generated at a node without a slot in that frame is *delayed* to the next round. Results are averages over the same 50 random networks of 100 nodes at density $\sigma = 10$ ($r = 50$ m, $a = 280$ m), simulated for 250 rounds after a warm-up of 50; $N = 100$, $\sigma = 10$ and $p = 0.2$ form the *reference point* of Table IX. All schedules use Rule B.

Besides BSPS and BAPS, the figures show the random static and random dynamic baselines of [1] (Rand-ST, Rand-DT); *Tree-ST*, a static schedule ordered along the tree as in the protocol, which represents the static routing-aware schedulers of [8]; the level thresholds of [11] (LT); the predictors D-BAPS and EWMA of Section VI-D; and an *oracle* that grants slots only to nodes that hold readings and to their relays. The 95% confidence intervals are given in the captions or are narrower than the markers.

A. Attribution of the Reported Gains

The protocol differs from its baselines in three respects. It uses Rule B, orders slots along the tree, and grants slots

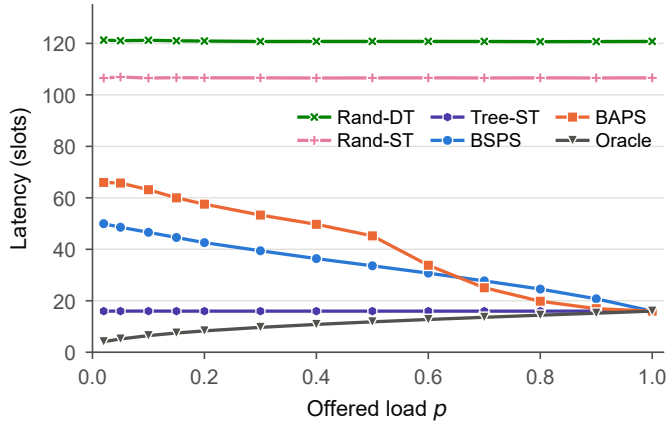


Fig. 7. Average latency against offered load ($N = 100$, $\sigma = 10$, 50 networks). The dynamic protocol (BSPS, BAPS) is faster than the random baselines but slower than the static schedule with the same tree ordering (Tree-ST); only the oracle is faster. The 95% confidence intervals are below ± 0.8 slots, except for Rand-ST and Rand-DT (± 6.4), whose latency depends on the random order drawn for each network.

only to nodes predicted to be active. Rand-DT changes only the slot order and is likewise not traffic-adaptive (Section IV-B), so a gain over these baselines cannot be attributed to adaptation. Tree-ST, which orders slots as the protocol does but never adapts, isolates the effect of ordering.

At $p = 0.2$ (Fig. 7), BSPS and BAPS deliver readings in 42.6 and 57.5 slots on average, against 106.6 for Rand-ST and 120.9 for Rand-DT. Both variants are thus faster than the random baselines, as reported in [1] (Section V-F). Tree-ST, however, has a latency of 16.0 slots at every load. The gain over the baselines results from the ordering. Under a random order a reading waits for the next round whenever the parent's slot precedes the child's (Section III-E), which never occurs under Tree-ST. With frames back to back, as in [1], a reversed hop costs one frame instead of a round; the differences decrease, but the ranking remains the same (Table VIII). Adaptation shortens the frame from 19.1 slots under Tree-ST to 15.1 under BSPS and 13.0 under BAPS, but adds latency (Section VI-B).

The third difference is the interference rule. The protocol description of [1] allows nodes to share a slot if they are not within two hops of each other in the routing tree. Read literally, as the *tree-distance criterion* of Section III-C, this rule allows even neighbors in the network to share a slot whenever they are more than two hops apart in the tree. On 50 networks at each of eight densities from 6 to 20, 39–63% of the transmissions scheduled with this criterion failed, depending on the density, whereas Rules A and B produced no failures. Rule B itself shortens the frame by 11–18% relative to Rule A (Section VIII-A).

B. Re-Probing and the Bound on Slot Removal

Because a node without a slot cannot report pending data, the protocol grants a slot again to every node denied in the previous frame (the *re-probe* rule, condition 1 of

Section V-B). This guarantee limits the number of slots that can be removed.

Proposition 1: Under the re-probe rule, no node is denied a slot in two consecutive frames. A node that never generates readings and has no child holding a slot is eventually denied in every second frame, under BSPS, BAPS or any other predictor that classifies a node as inactive once it stops using its slots; at most half of its slots are removed. The first reading generated after such an idle period, at a time independent of this alternation, falls in a frame in which the node holds no slot with probability one half. If a node is instead re-probed after $k - 1$ consecutive denials, up to a fraction $1 - 1/k$ of its slots is removed, and a reading may wait up to $k - 1$ frames.

Proof: A denied node is in D and receives a slot in the next frame. An idle node never uses a slot, so its predictor eventually classifies it as inactive, and condition 3 never applies. When it is denied, condition 1 grants it a slot in the next frame; it does not use that slot, is no longer in D , and is denied again. The pattern repeats with period 2, so a reading that arrives independently of the pattern falls into a denied frame with probability one half. With re-probing after $k - 1$ denials, the node holds one slot every k frames. $\square$

Without traffic every predictor removes 50.0% of the idle slots at the leaves, and the oracle 100% (Fig. 8(a)). With traffic the share can exceed one half, up to 62% under BAPS at $p = 0.4$, because a reading that arrives in a denied frame is transmitted in the next, granted frame, which is therefore not empty. For a leaf that alternates, a denied frame is empty with probability $1 - p$ and a granted one with probability $(1 - p)^2$, so the removed share of empty frames is $1/(2 - p)$, 62.5% at $p = 0.4$; Proposition 1 concerns a node that remains idle.

The alternation delays readings (Fig. 8(b)). Under light load 38–50% of all readings are delayed, half under BAPS and fewer under the other predictors, because a node also holds a slot whenever one of its children does. Each delayed reading waits one sampling period, so the predictor determines the number of delayed readings and the time model the length of the delay. At $p = 0.2$, 30% of the readings are delayed under BSPS and 42% under BAPS, which adds 26.6 and 41.6 slots to the latency of Tree-ST. The fraction falls as the load rises, since active nodes retain their slots by using them.

Re-probing less often removes more slots, 66.4%, 74.8% and 83.2% of those of idle nodes for $k = 3, 4$ and 6, but at $p = 0.1$ it raises the fraction of delayed readings under BSPS from 34% ($k = 2$) to 64% ($k = 6$) and the latency from 46.2 to 191.4 slots. The bound of one half thus follows from the choice $k = 2$, which also guarantees that no node waits more than one frame for a slot, so that no node can be starved, a property that [1] does not state.

C. Unbounded Memory of BAPS

BAPS compares a node's usage count with half the number of all previous frames, so the weight of past frames never decreases. This has two consequences.

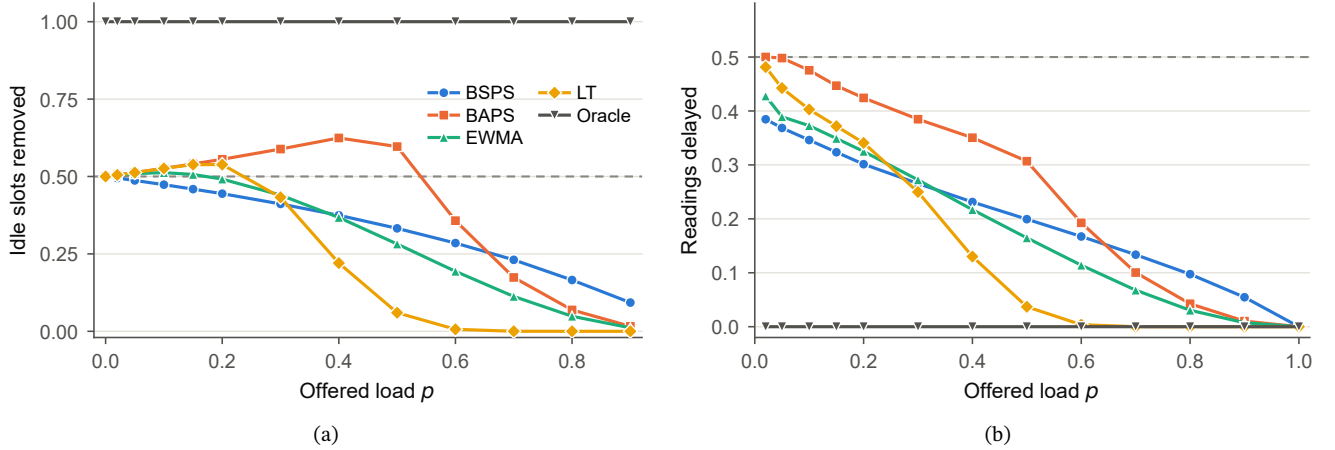


Fig. 8. Slot removal and delayed readings against the offered load ($N = 100$, $\sigma = 10$, 50 networks). (a) Idle slots removed, as the fraction of frames without data at a leaf in which the leaf holds no slot. (b) Fraction of readings delayed, that is, generated at a node that holds no slot in that frame. The dashed lines mark one half (Proposition 1). Same legend in both panels.

Proposition 2: Suppose a node holds a slot in every frame, and uses it with frequency $\rho_1 > 1/2$ until frame f_0 and with frequency $\rho_2 < 1/2$ afterward. Its expected usage count remains at or above half the number of frames until frame

$$f^* = f_0 \frac{\rho_1 - \rho_2}{1/2 - \rho_2}, \quad (6)$$

so BAPS continues to grant it a slot for approximately $f_0(\rho_1 - 1/2)/(1/2 - \rho_2)$ frames after its demand decreases. For $\rho_1 = 1$ and $\rho_2 = 0$, a node that was active for f_0 frames retains its slot for another f_0 .

Proof: While the node holds a slot in every frame, its expected usage count at frame $f > f_0$ is $\rho_1 f_0 + \rho_2(f - f_0)$. BAPS grants a slot while the count is at least $f/2$; equating the expected count with $f/2$ gives (6). $\square$

Proposition 3: If a node's usage count falls below half the number of previous frames and the node then generates a reading in every frame, BAPS grants it a slot in every second frame thereafter, unless one of its children holds a slot.

Proof: The node is denied, because its count is below the threshold, and granted a slot in the next frame by condition 1, which it uses. Over these two frames its count and the threshold $f/2$ both grow by one, so the deficit is unchanged and the node is denied again. The pattern repeats indefinitely. $\square$

We call the situation of Proposition 3 *persistent half-rate service*. It affects the event-driven nodes for which dynamic TDMA is intended, since a node that was inactive for a long time and then detects an event is served in only half of the frames while the event lasts. It has two causes acting together. BAPS records whether a node *used* a slot, not whether it had data to transmit, so a denied node records a zero regardless of its demand; and it weighs all past frames equally, so the deficit accumulated while the node was inactive never decreases.

D. Bounded-Memory Predictors

Either cause can be removed independently. LT [11] removes the first by computing its usage ratio over allocated slots only, so the count and the total increase together while the node is served. We propose to remove the second with decaying weights. D-BAPS maintains a decayed usage sum $S \leftarrow \lambda S + u$ and a decayed frame count $W \leftarrow \lambda W + 1$, both updated in every frame with $u = 0$ when the node holds no slot, and grants a slot when $S \geq W/2$. Each past frame thus weighs λ times as much as the frame after it, and the memory is approximately $1/(1 - \lambda)$ frames. For $\lambda = 1$ the rule is BAPS; for $\lambda < 1/2$ it is BSPPS, since a single unused frame then outweighs all earlier ones.

Proposition 4: Let $1/2 < \lambda < 1$ and let the history be long. Under D-BAPS, a node that used its slot in every frame and then ceases to use it retains the slot for $1 + \lfloor \ln 2 / \ln(1/\lambda) \rfloor$ frames. A node that was idle and then generates a reading in every frame is served in every second frame until it has used $\lfloor \ln((1 - \lambda)/2) / (2 \ln \lambda) \rfloor$ slots, and in every frame afterwards.

Proof: For a long history, $W = 1/(1 - \lambda)$. In the first case $S = W$ before the change, and after j unused frames S has decreased by the factor λ^j , so the node holds a slot as long as $\lambda^j \geq 1/2$. In the second case $S = 0$ before the change; the node then alternates between denied frames ($u = 0$) and re-probed frames ($u = 1$), so after n used slots $S = 1 + \lambda^2 + \dots + \lambda^{2(n-1)}$ and $S/W = (1 - \lambda^{2n})/(1 + \lambda)$, which reaches $1/2$ once $\lambda^{2n} \leq (1 - \lambda)/2$. $\square$

Both times are bounded and independent of the operating time of the network. With $\lambda = 0.9$, a node releases its slot after 7 frames and is served in every frame again after 15 used slots, approximately 29 frames, so D-BAPS makes the half-rate service transient. Removing both causes yields an *exponentially weighted moving average* (EWMA) of usage over the frames in which the node holds a slot. Our EWMA predictor maintains $\bar{u} \leftarrow 0.7 \bar{u} + 0.3 u$, updated only in such frames, and grants a slot when $\bar{u} \geq 0.5$; its decay corresponds to $\lambda = 0.7$. Proposition 1 applies to D-

BAPS and EWMA as to every other predictor.

Fig. 9 confirms Propositions 2–4. In panel (a), once the idle leaf becomes active, BSPS serves it in every frame immediately, EWMA after two frames and D-BAPS after 28, whereas BAPS never serves it in more than half of the frames. LT requires 152 frames, because its usage ratio, computed over the approximately 50 unused slots that the leaf held while idle, starts at zero. In panel (b), BAPS retains the slot for $f_0 + 1$ frames, one more than (6) because its threshold test includes equality, and LT for $0.67f_0$, whereas BSPS, EWMA and D-BAPS release it after one, two and at most seven frames, regardless of the history. With random usage ($\rho_1 = 0.8$, $\rho_2 = 0.1$), 13–15% of the runs fall below the threshold during the active phase and are denied the slot as soon as demand decreases, which lowers the mean release time; BAPS can thus reduce even a node that is active 80% of the time to half-rate service.

E. Cost of Schedule Distribution

Rebuilding the schedule at every frame requires communication in both directions. The sink requires the slot usage of every node, which incurs only a small overhead, since a parent observes whether its child transmitted and can add one bit to its own packet; every node must also receive the new schedule, which requires at least one reception. Removing an idle slot saves at most $47.9 - 4.9 = 43 \mu\text{C}$, at the listening parent (Table I). Since no node is denied a slot in two consecutive frames (Proposition 1), prediction saves at most approximately $21.5 \mu\text{C}$ per node per frame, whereas one reception of a schedule update costs $72.1 - 4.9 = 67.2 \mu\text{C}$.

Fig. 10 compares three ways of distributing the schedule (Section VII-E). With *free updates*, BSPS saves $19.3 \mu\text{C}$ per node per round against Tree-ST at $p = 0$ and $14.2 \mu\text{C}$ at $p = 0.2$. With *ACK updates*, in which a node that transmits obtains its next slot from the acknowledgment and every other node requires one reception, BSPS consumes $47.9 \mu\text{C}$ more than Tree-ST at $p = 0$, 30.3 at $p = 0.2$ and 1.3 at $p = 0.9$. This model is optimistic, since an acknowledgment could carry the next schedule only if that schedule were computed a frame in advance, from older information. With *broadcast updates*, in which every node receives the schedule and every parent forwards it, the excess is 71.0 – $76.2 \mu\text{C}$ for $p \leq 0.2$. The energy argument for rebuilding the schedule therefore holds only if updates have negligible cost; [1] does not model this cost, and [5] explicitly notes it. The PriorityScore of [11], which requires four measured values from every node in every frame, increases it further.

F. Methodological Issues

Several aspects of the published evaluation also limit its conclusions.

- *Parameters.* With $a = 540$ m, $r = 50$ m and 20 to 200 nodes, (2) gives σ between 0.5 and 5.4, not the stated $[2, 20]$, which requires varying a , as in [8].
- *Connectivity.* At low density many nodes have no path to the sink; in our 50 networks of 100 nodes, 6% had

TABLE V
SUMMARY OF THE CRITICAL ANALYSIS

Problem	Finding	Evidence
Attribution	The latency gain over the baselines results from tree ordering, in both time models; adaptation shortens the frame but delays a fraction of the readings by one sampling period	Fig. 7
	The <i>tree-distance criterion</i> , the literal reading of the rule in [1], causes 39–63% of transmissions to fail	Sec. VI-A
Re-probing	With re-probing after one denial, an idle node retains a slot in every second frame, and up to half of the readings are delayed by one sampling period; less frequent re-probing removes more slots but increases the delay	Prop. 1, Fig. 8
Memory	BAPS releases a slot after a time proportional to the node's history and can serve an active node in only half of the frames, indefinitely; decaying weights (D-BAPS) bound both times, and EWMA eliminates the half-rate service	Props. 2–4, Fig. 9
Update cost	When every node must receive the rebuilt schedule, the dynamic protocol consumes more energy than static TDMA at every load below $p = 1$	Fig. 10

a path to it at $\sigma = 2$, 45% at $\sigma = 4$ and 86% at $\sigma = 6$, and [1] does not state whether unreachable nodes are included in its averages.

- *Energy and duty cycle.* No energy model or unit is given, and the duty cycle is normalized by the frame length, so a shorter frame can increase it although the radio is on for less time.
- *Traffic and statistics.* No traffic model is specified, although the predictors are designed to anticipate traffic, and no numerical values or confidence intervals are reported.
- *Inconsistencies.* In the example of [1] (its Table I), nodes 4 and 6 share a slot although they are two hops apart. In [11], the level-order traversal enqueues the children of leaves, and the threshold formula assigns K_{top} to the leaves; we implemented the behavior described in its text (Section V-G).

Table V summarizes these findings.

VII. SIMULATION METHODOLOGY

We implemented a discrete-time simulator in Python for this study, compact enough to be described in full here. Its unit tests reproduce every hand-worked result of Sections III–V, namely the routing tree, the schedule of Table II, the average latency of 3.5 slots, the 4 and 9 slots of Fig. 6, and the hidden-terminal collision of Fig. 2(b). They also compare the simulator with results derived by hand, namely the latency of random schedules from (3)–(4), the fraction of delayed readings at a BSPS leaf, the charge of the static and oracle schedules, the two limits of D-BAPS, and the times of Proposition 4. Table VII lists the parameters.

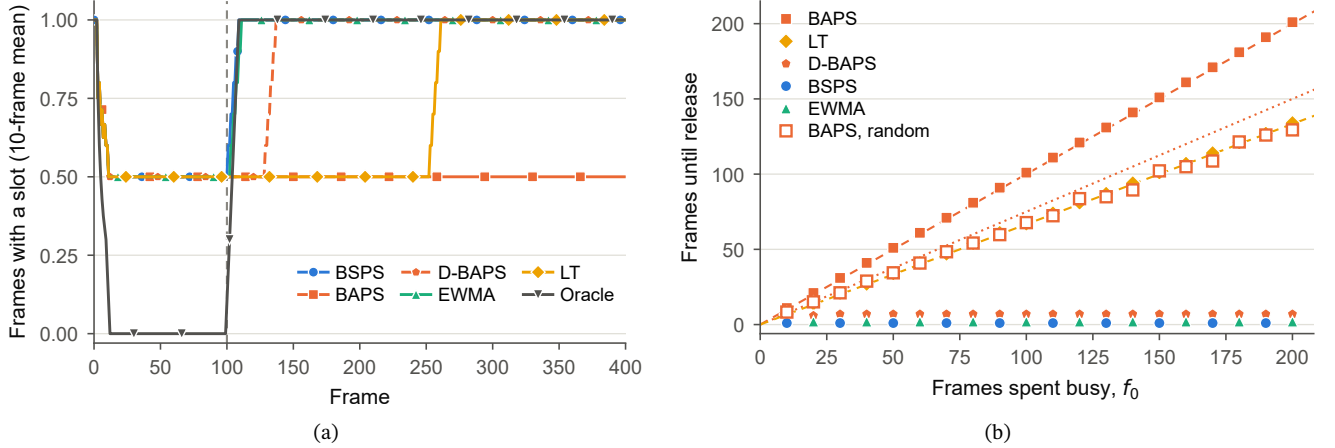


Fig. 9. Response of the predictors to changes in demand at a single leaf. (a) Fraction of frames in which a leaf holds a slot, averaged over ten frames, when the leaf is idle until frame 100 (dashed line) and generates a reading in every frame afterward. (b) Number of frames for which a leaf retains its slot after being active for f_0 frames. The filled markers show $\rho_1 = 1$ and $\rho_2 = 0$, with dashed lines for (6) (BAPS) and $f_0(1/K_{\text{leaf}} - 1)$ (LT); the hollow squares show BAPS with $\rho_1 = 0.8$ and $\rho_2 = 0.1$ (mean of 400 runs), with a dotted line for (6) at these values. D-BAPS uses $\lambda = 0.9$, LT the threshold $K_{\text{leaf}} = 60\%$ of the deepest level.

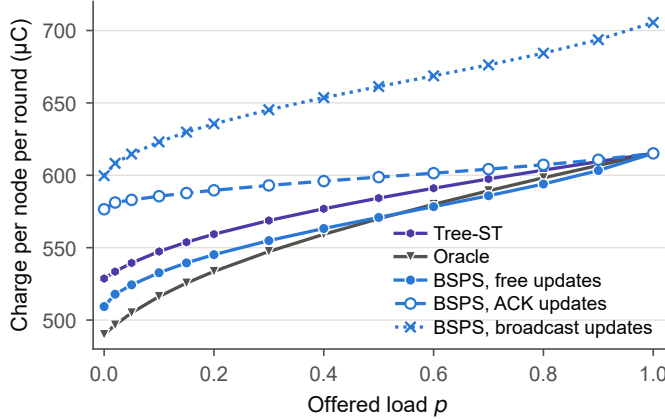


Fig. 10. Charge per node per round of 100 slots ($N = 100$, $\sigma = 10$, 50 networks) under three ways of distributing the schedule (Section VII-E). With free updates BSPS saves energy against the static schedule (Tree-ST); when every node must receive the new schedule, through the acknowledgment of its transmission or a broadcast, BSPS consumes more than Tree-ST at every load below $p = 1$.

A. Networks

Nodes are placed uniformly in a square with the sink at its center [1] and connected under the UDG model with $r = 50$ m; the density is set through the side length a , as in [8]. Only the sink's connected component is scheduled, over the minimum-hop tree of Section III-A. Every experiment at the reference point ($N = 100$, $\sigma = 10$) uses the same 50 networks, in which 98.9% of the nodes are connected to the sink on average. The density sweep ($\sigma = 6$ to 20, $N = 100$) and the size sweep ($N = 25$ to 200, $\sigma = 10$) use 40 other networks per point. At $\sigma = 6$ and 8 many of these networks are not fully connected, and the sink's component holds only 88 and 96 nodes on average, so the low-density points also describe smaller networks.

B. Time and Traffic

Time is divided into slots of 15 ms. A *frame* is the sequence of slots of one schedule, and a *round* is an interval of 100 slots (1.5 s), similar to the repeating slotframe of TSCH [2]. Every TDMA frame starts at the beginning of a round, and the radios sleep for the rest of it, so the numbers of frames and rounds coincide. The network samples once per round, and a reading that arrives after a slot on its path waits for that slot in the next round, one *sampling period* later; in the second case of (4), the frame length L is replaced by the round length. Benrebbouh and Louail run frames back to back [1], so their sampling period is one frame. With a fixed round, the sampling period is equal for all schemes, the energy per round is comparable, and a comparison with duty-cycled CSMA, which has no frames, is possible. To assess the effect of this choice, the reference point is also simulated with frames back to back and with rounds of 30 to 400 slots (Section VIII-E).

We use three traffic models. Under *memoryless* traffic, each node generates a reading at the start of every round with probability p , independently of the past, so whether a node used its slot in the last round carries no information about whether it has a reading in the current round. Under *on/off* traffic, each node alternates between active runs, in which it generates a reading in every round, and inactive runs, in which it generates none. Run lengths are random (geometric), with mean $\bar{\ell}$ rounds when active and $\bar{\ell}(1-p)/p$ when inactive, so that the long-run load remains p . For $\bar{\ell} = 1/(1-p)$ an active node remains active with probability p and an inactive node becomes active with probability p , which is memoryless traffic; a larger $\bar{\ell}$ describes the event-driven activity for which dynamic TDMA is intended. Under *intermittent* traffic, an active node generates a reading only with probability $q < 1$ in each round of its run, and inactive runs are shortened to a mean of $\bar{\ell}(q-p)/p$ rounds to maintain the load p .

TABLE VI
SIMULATED SCHEMES

Scheme	Rule	Slot order	Slot allocation
Rand-ST [1]	A or B	Random, fixed	Every node
Rand-DT [1]	B	Random, per frame	Every node
Tree-ST	B	Tree	Every node
BSPS, BAPS [1]	B	Tree	Predicted (Alg. 1)
LT [11]	B	Tree	Predicted, by level
D-BAPS (ours)	B	Tree	Predicted, decay λ
EWMA (ours)	B	Tree	Predicted, held slots
Oracle	B	Tree	Nodes with data, relays
CSMA	n/a	Contention	None (reference)

Readings wait in the node's queue, and a node sends all queued readings in one packet (aggregated convergecast, Section III-B).

C. Schedulers

Table VI lists the schemes; all TDMA schemes except the first use Rule B. The tree-ordered schemes process the tree from the leaves and search for a slot only after those of the node's children, without wrapping around the frame, so every child transmits before its parent, as [1] states ("we gave each son a slot prior to that of his father"); the search of [8] may wrap around and violate this order. The dynamic schemes start with three static frames, as BAPS requires, re-probe a node denied in the previous frame ($k = 2$), and apply relay protection (condition 3 of Section V-B). LT implements only the level thresholds of [11], decreasing linearly from 60% at the deepest level to 30% at depth 1, without the PriorityScore and variable slot lengths. EWMA uses $\bar{u} \leftarrow 0.7\bar{u} + 0.3u$ and the threshold 0.5, and D-BAPS uses $\lambda = 0.9$. In Section VIII-D, λ varies from 0 to 1 and EWMA uses the same decay, $\bar{u} \leftarrow \lambda\bar{u} + (1 - \lambda)u$; since a memory of 100 frames converges slowly, those runs use a warm-up of 400 rounds. The oracle grants slots only to nodes with queued readings and to their relays.

D. Contention-Based Reference

The CSMA reference uses the listen/sleep cycle of S-MAC [4]. Every round begins with an active window in which all nodes are awake and contend, and the radios sleep for the rest of the round. The window is as long as the Tree-ST frame of the same network, so that CSMA and TDMA use the same channel time. Contention is a simplified form of the IEEE 802.15.4 procedure. A node with data attempts a transmission in the next slot, defers if a neighbor is already transmitting, backs off for a random number of slots below a contention window that doubles from 1 to 8, and drops its readings after four failed attempts. Hidden nodes may collide, and a transmission fails under the same protocol model as in TDMA.

E. Energy and Metrics

Energy is charged per slot with the measured values of Table I. In TDMA, a node that transmits in its slot is

TABLE VII
SIMULATION PARAMETERS

Parameter	Value
Nodes N ; density σ	100 (25–200); 10 (6–20)
Range r ; networks per point	50 m; 50 (40 in the sweeps)
Offered load p	0.2 (0–1) per node per round
Traffic	Memoryless, on/off, intermittent
Active run $\bar{e}$; q	1.25–50 rounds; 0.5–1
Slot; round	15 ms; 100 slots (also 30–400)
Rounds per run	50 (400) warm-up, 250, 40 drain
Energy per slot	Table I, GINA mote [2]
Dynamic schemes	3 static frames; re-probe after 1 denial
D-BAPS decay λ	0.9 (0–1)
CSMA window; retries	Tree-ST frame; 3

charged a transmit slot and its parent a receive slot, or an idle-listening slot if the child has no data; an owner without data sleeps [2]. Acknowledgments are not charged in TDMA, since they cost every TDMA scheme the same per transmission; charging them would add $17\mu\text{C}$ per node per round at $p = 0.2$. Three models charge the distribution of the schedule for the dynamic schemes. *Free updates* add no charge; with *ACK updates*, a node that transmitted obtains its next slot from the acknowledgment and every other node is charged one receive slot; with *broadcast updates*, every node is charged one receive slot and every parent one transmit slot. Unless stated otherwise, updates are free. In CSMA, an exchange costs a transmit or receive slot with acknowledgment, a node whose neighbor transmits is charged a receive slot, and any other awake node an idle slot. Since the radio is on for only a 2.6 ms guard time in the idle slot of Table I, this charge is a lower bound for CSMA, whose nodes listen for the whole window. All remaining slots of the round are sleep slots.

We report the average latency, in slots from the start of the round in which a reading is generated to the slot in which it is delivered to the sink, and its 95th percentile, computed per network and averaged; the fraction of delayed readings; the frame length; the charge per node per round; and the delivery ratio. Each run lasts 250 rounds after a warm-up of 50 (400 in the memory sweep), plus 40 rounds for late readings to arrive.

VIII. RESULTS

We vary the density, the network size, the offered load, the persistence of node activity, the memory of the predictors and the sampling period. Table IX lists all schemes at the [reference point](#) on the 50 networks of Section VI. The density and size sweeps use their own networks, so their values at $\sigma = 10$ and $N = 100$ differ slightly from Table IX, within the confidence intervals.

A. Density

Fig. 11(a) shows the frame length against the density. In a denser network each node has more neighbors, hence more conflicts, and the nodes near the sink have more

children; between $\sigma = 6$ and 20, the lower bound $\Delta(T)$ of Section III-D rises from 7.6 to 21.0 slots and the Tree-ST frame from 15.9 to 30.9 slots, 2.1 to 1.5 times the bound. Three design choices affect this length. Under Rule B the random static frame is 11–18% shorter than under Rule A, because Rule B allows two nodes two hops apart to share a slot when their common neighbor is the parent of neither (Section III-C). Placing every child before its parent requires additional slots; Tree-ST frames are 14–39% longer than Rand-ST frames under the same rule, so at $\sigma = 10$ the ordering offsets the gain of Rule B. Adaptation shortens the frame, BPS by 13–28% and BAPS by 25–40%, more in denser networks. The frame of the oracle has 10.5 to 12.4 slots at every density, since it assigns slots only to nodes that hold readings and to the relays on their paths. The dynamic frames may fall below $\Delta(T)$, which bounds only schedules that assign every node a slot.

Fig. 11(b) shows latency on a logarithmic scale, since the schemes differ by more than an order of magnitude. The random orders are the slowest, and their latency decreases as the network becomes denser; Rand-ST has a latency of 181 slots at $\sigma = 6$ but 58 at $\sigma = 20$. Their latency is determined by the depth of the tree, not by the frame, since every hop at which the parent's slot precedes the child's adds an entire round to the latency, and the deepest node is 9.9 hops from the sink at $\sigma = 6$ but 3.9 at $\sigma = 20$. The latency of Tree-ST increases with its frame length, from 13.8 to 25.0 slots, because every reading is delivered to the sink within the frame in which it is generated. BPS and BAPS remain 25 to 45 slots above Tree-ST at every density (38.6–51.4 and 51.7–70.0 slots), because 27–34% of the readings under BPS and 39–46% under BAPS are delayed. The oracle remains at approximately 9 slots.

CSMA, shown for reference, has a latency of 46.3 slots at $\sigma = 6$ and 15.3 at $\sigma = 20$, below Tree-ST for $\sigma \geq 16$, because its paths become shorter and its window, equal to the Tree-ST frame, longer; at every density 11–15% of its readings are lost.

B. Network Size and Offered Load

In Fig. 12(a), the number of nodes varies from 25 to 200 at constant density, so a larger network covers a larger area. The tree deepens from 2.9 to 8.5 hops, while $\Delta(T)$ grows only from 9.0 to 11.7 slots and the Tree-ST frame from 12.6 to 21.8 slots. The latency of Tree-ST increases with its frame length, from 9.7 to 19.0 slots, and that of the oracle rises from 3.8 to 11.2. The latencies of BPS and BAPS change only slightly (40.7–44.5 and 57.0–58.3 slots), since approximately the same share of readings is delayed at every size (29–34% under BPS) and each delayed reading costs one round regardless of the network size. The latencies of the random baselines (not drawn) increase with depth, Rand-ST from 34 to 175 slots; at 25 nodes they are faster than BPS and BAPS, because a shallow tree has few hops that can be reversed. As a function of the offered load (Fig. 12(b)), the TDMA schemes behave as in Fig. 7.

Duty-cycled CSMA is included as a reference outside TDMA. Under light load it is the fastest scheme after the

oracle (4.8 slots at $p = 0.02$), since only nodes with data contend, and it is faster than Tree-ST up to $p = 0.1$ and than BPS up to $p = 0.2$. As the load and the network size increase, collisions and backoffs accumulate. At $p = 1$ the latency of CSMA is 124 slots and 42.5% of the readings are lost, and at 200 nodes the latency is 48.4 slots and 22.0% are lost, whereas the TDMA schedules deliver every reading in our model. In all sweeps it consumes 2.1 to 3.8 times the charge of Tree-ST (Fig. 12(c)), because every node listens during the whole window, whereas a TDMA node is active only in its own slot and in those of its children.

Every scheme draws at least 490 μC per round, the charge of 100 sleep slots, and the MAC protocol determines only the charge above this minimum; the TDMA schemes lie within 5% of each other. With a 2000 mAh battery, (1) converts the charges of Table IX into lifetimes of 224 days for Tree-ST, 229 for BPS with [free updates](#), 212 for BPS with [ACK updates](#), 234 for the oracle and 84 for CSMA.

C. Bursty Traffic

Under memoryless traffic a node's past activity carries no information about its current demand, so every predictor delays readings at the rate of Section VI-B. In the events that motivate dynamic TDMA, however, a node near an event remains active for several consecutive rounds. In Fig. 13 the long-run load remains $p = 0.2$, but every node alternates between active and inactive runs (Section VII-B), with a mean active run $\bar{\ell}$ from 1.25 rounds (memoryless traffic) to 50 rounds.

BPS benefits most, because it uses only the last frame; once a node has used its slot, it retains it for the rest of its active run. Its fraction of delayed readings falls from 30% to 0.8%, and its latency from 42.7 to 13.2 slots. BPS is within 0.3 slots of Tree-ST (16.0 slots) at $\bar{\ell} = 10$ and faster for $\bar{\ell} \geq 20$ (14.4 slots), since the readings it serves are delivered within a shorter frame. EWMA requires several frames to update its average and adapts more slowly; its delayed readings fall from 33% to 2.3% and its latency to 14.6 slots. The latency of BAPS does not decrease. At every run length it delays 38–44% of the readings and has a latency of 52–59 slots, because a node that is active 20% of the time has a usage ratio over its entire history below one half, so the node receives [half-rate service](#) in every active run. LT, whose usage ratio also covers the whole history, delays 24–35%. CSMA and the oracle do not depend on the run length.

The charge changes only slightly with the run length. BPS saves 12–15 μC per node per round relative to Tree-ST, approximately half of the oracle's 25.7 μC , because the [re-probe rule](#) still grants every idle node a slot in every second frame; with ACK updates it consumes 30–31 μC more than Tree-ST at every run length. Once runs last two rounds or more, BAPS consumes slightly less than the oracle, because the readings it delays are later sent together in one packet. Persistent activity thus eliminates the latency added by prediction, while the bound on its energy saving and the cost of distributing the schedule remain.

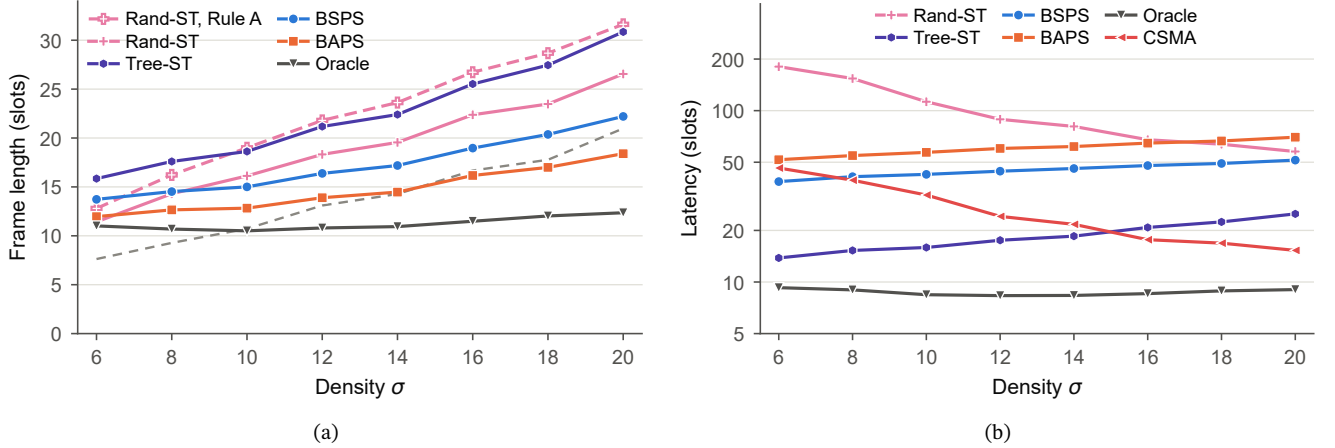


Fig. 11. Effect of density ($N = 100$, $p = 0.2$, 40 networks per point). (a) Frame length; the dashed line is the lower bound $\Delta(T)$ for schedules that assign every node a slot. (b) Average latency, on a logarithmic scale. The 95% confidence intervals are below ± 1.1 slots in (a) and below 10% of the mean in (b).

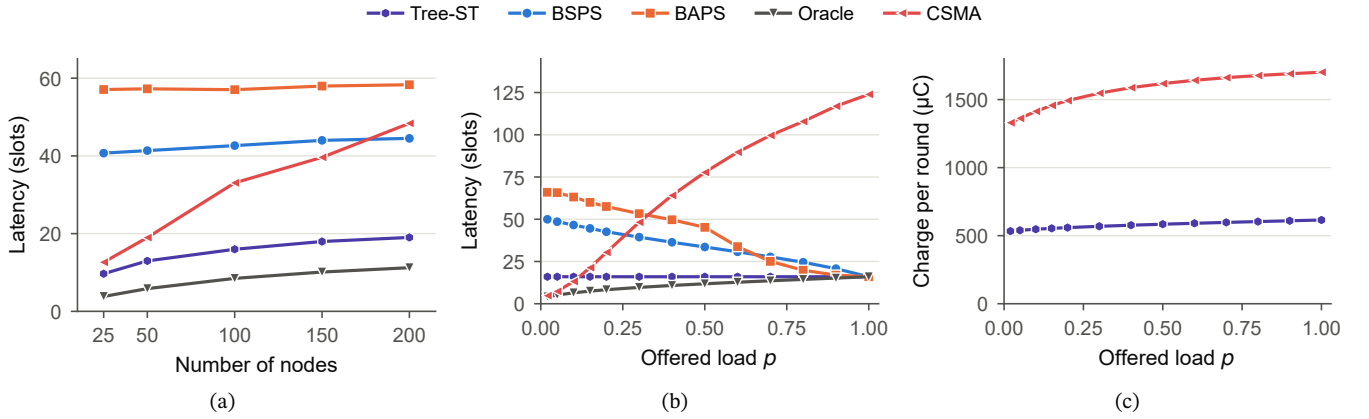


Fig. 12. Network size and offered load, with duty-cycled CSMA as a reference. (a) Average latency against the number of nodes ($\sigma = 10$, $p = 0.2$, 40 networks per point). (b) Average latency and (c) charge per node per round against the offered load ($N = 100$, $\sigma = 10$, 50 networks); in (c) the other TDMA schemes lie within 7% of Tree-ST and are not drawn. The 95% confidence intervals are below ± 3.5 slots and $\pm 30 \mu\text{C}$.

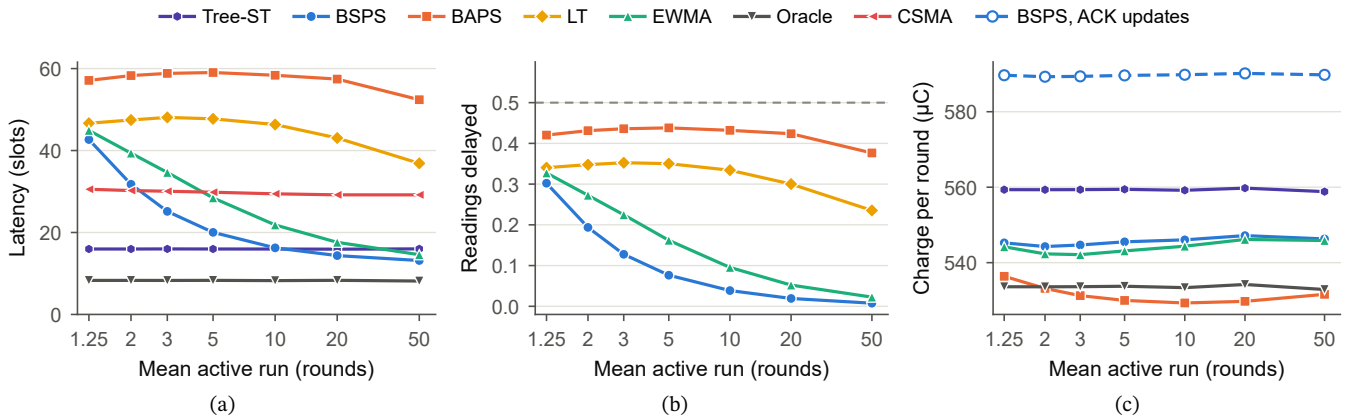


Fig. 13. On/off traffic ($N = 100$, $\sigma = 10$, long-run load $p = 0.2$, 50 networks) against the mean length $\bar{\ell}$ of an active run, from 1.25 rounds (memoryless) to 50, on a logarithmic scale. (a) Average latency. (b) Fraction of readings delayed; the dashed line marks one half. (c) Charge per node per round; CSMA (approximately $1490 \mu\text{C}$) is not drawn. The 95% confidence intervals are below ± 2.0 slots, ± 0.012 and $\pm 1.1 \mu\text{C}$.

D. Memory Length of the Predictors

Fig. 14 evaluates D-BAPS, proposed in Section VI-D, varying the memory $1/(1 - \lambda)$ of D-BAPS from one frame ($\lambda = 0$, BSPS) to infinity ($\lambda = 1$, BAPS), with the same

decay for EWMA. The traffic at $p = 0.2$ is memoryless, in steady active runs of $\bar{\ell} = 20$ rounds, or in intermittent runs of the same length in which an active node reports in 70% of the rounds ($q = 0.7$).

Under memoryless traffic every memory longer than one frame raises latency, since past usage carries no information about current demand. Under steady runs a memory of one frame is also optimal, with BSPS at 14.3 slots, less than Tree-ST; every longer memory responds more slowly to the start and end of a run. Under intermittent runs a short memory reduces the latency of EWMA. With $\lambda = 0.7$ its latency is 25.1 slots and it delays 12.6% of the readings, compared with 27.4 slots and 15.0% for BSPS, because a single round without a reading after several with readings does not cause the node to be denied a slot. D-BAPS does not improve on BSPS under any of the three traffic types. At $\lambda = 0.7$ its latency is 30.3 slots, because it still counts the frames without a slot as unused, so every node that becomes active first receives the transient half-rate service of Proposition 4. At the reference point, D-BAPS with $\lambda = 0.9$ delays 41% of the readings, close to the 42% of BAPS (Table IX).

The advantage of EWMA depends on the probability q that an active node reports in a round. With $\bar{\ell} = 20$, EWMA with $\lambda = 0.7$ is faster than BSPS by 1.5 to 2.5 slots for $0.6 \leq q \leq 0.8$ and by 1.0 slot at $q = 0.9$, equally fast at $q = 0.5$, and slower by 3.2 slots at $q = 1$ (95% intervals below ± 0.2 for these paired differences). The optimal memory length thus depends on the gaps within an active run, and for $q < 1$ no predictor attains the latency of 16.0 slots of Tree-ST. Decaying weights bound the memory of BAPS without removing its half-rate service, and reduce latency only when the frames in which a node holds no slot are also excluded.

E. Sampling Period and Time Model

Table VIII gives the latency at the reference point with frames back to back, as in [1], and with rounds of 30, 100 and 400 slots. The ranking is the same in every case (the oracle, then Tree-ST, BSPS, BAPS, Rand-ST and Rand-DT), but the magnitudes differ. Tree-ST and the oracle never delay a reading and do not depend on the time model, whereas every other scheme incurs a delay of one sampling period for each delayed reading or reversed hop. With frames back to back, BSPS is 1.0 slot slower than Tree-ST and Rand-ST 8.1 slots slower; with rounds of 400 slots, 117 and 386. The relative energy saving decreases instead. BSPS saves $14 \mu\text{C}$ per node per round at every round length, which is 6.5% of the total with rounds of 30 slots but 0.7% with rounds of 400, because the sleep charge grows with the round length. Ratios of latencies therefore depend on the time model, so we quote differences in slots. BAPS was also sensitive to the operating time of the network; at $\bar{\ell} = 20$ its latency rose from 57.4 slots after a warm-up of 50 rounds to 59.7 after 400, while BSPS did not change.

F. Decomposition of the Differences

In the first five rows of Table IX, the design elements of the protocol are added one at a time (Section VI-A):

- Replacing Rule A by Rule B (Rand-ST) shortens the frame by 2.6 slots (14%) and the latency by 2.6 slots.

TABLE VIII
LATENCY (SLOTS) AT THE REFERENCE POINT UNDER OTHER TIME MODELS

Scheme	Back to back	Round length (slots)		
		30	100	400
Oracle	8.3	8.3	8.3	8.3
Tree-ST	16.0	16.0	16.0	16.0
BSPS	17.0	21.5	42.6	133.0
BAPS	18.1	27.8	57.5	184.8
Rand-ST	24.1	37.6	106.6	402.4
Rand-DT	27.3	42.5	120.9	456.8

TABLE IX
ALL SCHEMES AT THE REFERENCE POINT ($N = 100$, $\sigma = 10$, $p = 0.2$, MEMORYLESS TRAFFIC, 50 NETWORKS)

Scheme	Frame (slots)	Latency (slots)		Delayed (%)	Charge (μC)	Deliv. (%)
		Mean	P95			
Rand-ST, Rule A	19.1	109.2	229.6	0	559.3	100
Rand-ST	16.5	106.6	236.5	0	559.3	100
Tree-ST	19.1	16.0	19.1	0	559.3	100
BSPS	15.1	42.6	115.1	30	545.2	100
Oracle	10.4	8.3	11.6	0	533.6	100
Rand-DT	16.5	120.9	284.7	0	557.8	100
BAPS	13.0	57.5	118.9	42	536.2	100
D-BAPS	13.4	54.5	118.0	41	538.2	100
LT	14.7	46.6	116.0	34	542.7	100
EWMA	14.8	44.7	114.9	33	544.2	100
CSMA	19.1*	30.4	112.9	n/a	1492.6	86

*Listen window. P95 is the 95th percentile, per network, averaged.

Rule B except in the first row; free schedule updates. 95% interval

of the mean latency: ± 6.2 (Rand-ST), ± 5.3 (Rand-DT), $\leq \pm 1.5$ otherwise.

- Replacing the random order by tree ordering (Rand-ST to Tree-ST) reduces the latency by 90.6 slots with rounds of 100 slots, or by 8.1 slots with frames back to back, and lengthens the frame by 2.6 slots.
- Adding prediction (Tree-ST to BSPS) shortens the frame by 3.9 slots and reduces the charge by $14.2 \mu\text{C}$ with [free updates](#), but increases it by $30.3 \mu\text{C}$ with [ACK updates](#); the latency increases by 26.6 slots with rounds of 100 slots, or by 1.0 slot with frames back to back, because 30% of the readings are delayed. Under persistent activity this delay vanishes (Section VIII-C).
- Perfect knowledge of demand (BSPS to oracle) reduces the latency by 34.3 slots and the frame by 4.7 slots.

The other predictors (LT, EWMA and D-BAPS) fall between BSPS and BAPS. The 95th percentile confirms this for individual readings. Under Tree-ST no reading waits longer than one frame (19.1 slots), whereas under every predictor the slowest 5% of readings wait more than a round.

IX. DISCUSSION

A. Limits of Prediction

A predictor that cannot receive requests must continue to [re-probe](#) the nodes it has denied, and the re-probe period determines the trade-off between the idle slots it

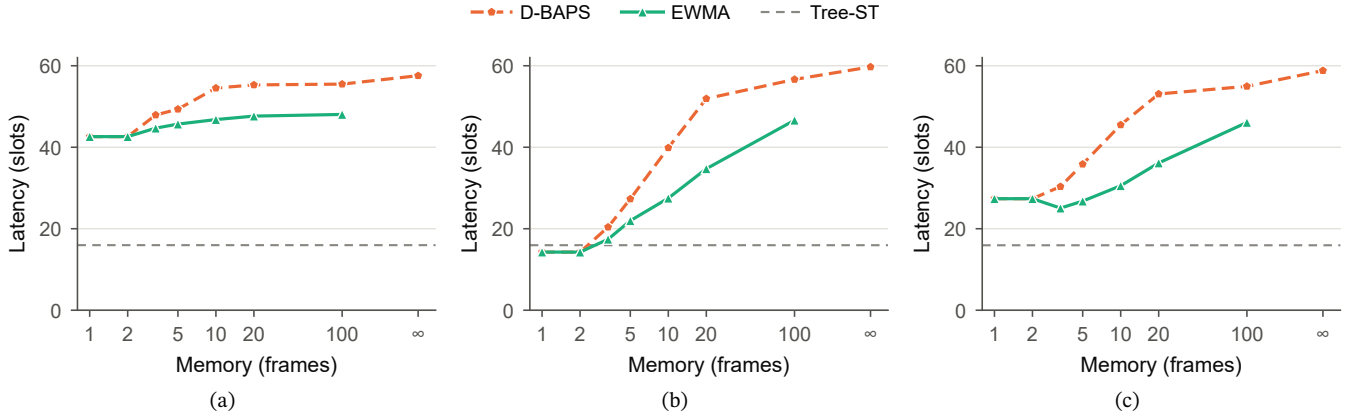


Fig. 14. Latency against the memory $1/(1-\lambda)$ of the predictor ($N = 100$, $\sigma = 10$, $p = 0.2$, 50 networks, warm-up of 400 rounds). D-BAPS counts every frame; EWMA counts only the frames in which the node holds a slot, so its memory is measured in such frames. (a) Memoryless traffic. (b) Steady active runs, $\bar{\ell} = 20$. (c) Intermittent runs, $\bar{\ell} = 20$, $q = 0.7$. A memory of one frame is BSPS; the last D-BAPS point (∞) is BAPS. The dashed line marks Tree-ST. The 95% confidence intervals are below ± 0.8 slots.

removes and the delay of readings that arrive at denied nodes (Section VI-B). The traffic determines the second limit. A predictor can only anticipate demand that persists, and its memory must match the gaps within an active run while ignoring the frames in which the node held no slot (Sections VIII-C and VIII-D). The time model determines the delay caused by each misprediction, which equals one sampling period.

B. Conditions Favoring Dynamic TDMA

Four conditions must be met simultaneously. First, node activity must persist over many frames, or prediction delays readings instead of serving them; in our networks BSPS matched the static tree-ordered schedule once activity lasted approximately ten rounds and achieved lower latency from approximately twenty rounds onward. Second, schedule changes must be distributed to the nodes at negligible cost, since the saving is small. Even the oracle saves only $25.7 \mu\text{C}$ per node per round at the reference point, 4.6% of the total, which extends the lifetime of a 2000 mAh battery from 224 to 234 days; BSPS saves approximately half of that, and receiving one schedule update per round costs a node more than this saving (Section VI-E). Third, the sampling period must be short, since a longer period lengthens the delay of each misprediction and reduces the relative saving (Section VIII-E). Finally, the application must tolerate a delay of one sampling period for part of its readings.

The frame length, the quantity most often reported for dynamic TDMA [1], affects latency and the channel time left to other traffic. It does not directly affect energy, since in a duty-cycled network a node sleeps outside its own and its children's slots regardless of the frame length; the energy saved by dynamic TDMA results from the idle listening that it removes at the parents.

Duty-cycled CSMA adapts to traffic without predicting it, since only nodes with data contend, and under light load its latency approaches that of the oracle. Its costs are idle listening during the whole active window and collisions

that grow with load and network size (Section VIII-B). This agrees with Rajendran et al., who found that schedule-based protocols “exhibit higher delays than contention-based” ones but deliver more packets [12], and with the Z-MAC measurements of Section II-D. Dynamic TDMA is intended for light load, where contention-based access already achieves low latency without prediction.

C. Limitations

The study inherits the assumptions of the studied protocol and adds several of its own.

- *Radio*. Interference follows the protocol model, links never fail except through collisions, clocks are perfectly synchronized, and a single channel is used, whereas real links are subject to fading and clock drift, and TSCH uses channel hopping [6]. In this model no reading is lost under TDMA, so the comparison of delivery ratios favors TDMA.
- *Network*. The routing tree is fixed, and neither its construction nor the collection of slot usage at the sink is charged.
- *Traffic*. Nodes generate readings independently of one another and at the start of a round; real events are correlated in space, and readings generated during a round would add a common delay to every scheme. All queued readings travel in one packet, so that a delayed node can send two readings in one transmission; this favors the dynamic schemes in energy and alone makes BAPS consume slightly less than the oracle under most on/off settings.
- *Energy*. The charges were measured on one platform with 15 ms slots. TDMA packets carry no acknowledgment in our model; acknowledgments would add $17 \mu\text{C}$ per node per round at $p = 0.2$ to every TDMA scheme.
- *Contention*. Our CSMA is a simple reference without RTS/CTS handshake or low-power listening. Its idle listening is charged at the guard-time cost of TSCH, so

its charge is a lower bound, and low-power listening would reduce the energy difference.

X. CONCLUSION

This report studied the cross-layer dynamic TDMA protocol of Benrebbouh and Louail [1]. Starting from the energy consumption of a sensor node, it formalized TDMA scheduling, placed the protocol among other TDMA schedulers, analyzed its mechanism, proposed a version of BAPS with decaying weights, and simulated the protocol against its baselines, a static tree-ordered schedule, other predictors and an oracle, with duty-cycled CSMA as a reference. Six conclusions follow.

- 1) The latency gain over random schedules results from placing every child's slot before its parent's, a static property of the schedule that requires no computation at run time, although it lengthens the frame by approximately 15% and makes the schedule dependent on the routing tree. At the [reference point](#) it lowers latency by 90.6 slots with rounds of 100 slots and by 8.1 slots with frames back to back.
- 2) With re-probing after one denial, an idle node retains a slot in every second frame, and up to half of the readings are delayed by one sampling period. Under memoryless traffic, no predictor is faster than the static schedule.
- 3) Prediction is beneficial when node activity persists. With steady active runs of twenty rounds or more, BAPS delivers readings up to 2.8 slots faster than the static schedule, but when active nodes report only intermittently, no predictor we tested matches the static schedule.
- 4) The [persistent half-rate service](#) of BAPS requires two features together, equal weights over all past frames and a zero for every frame without a slot. The decaying weights we propose (D-BAPS) make it transient; only a decaying average over the frames with a slot (EWMA) removes it, improving on BAPS by up to 2.5 slots when active nodes reported in 60–90% of the rounds.
- 5) The energy saved by prediction is small, 14 μC per node per round under BAPS at the reference point, approximately half of the oracle's 25.7 μC , and a realistic distribution of a rebuilt schedule consumes more charge than prediction saves. A longer sampling period reduces the relative saving and increases the latency added by prediction.
- 6) Duty-cycled CSMA, the reference outside TDMA, achieves lower latency under light load without any prediction, but at equal channel time it consumes at least twice as much energy as TDMA, and part of its readings are lost as the load and the network size increase.

Dynamic TDMA is therefore advantageous only with persistent traffic, low-cost schedule distribution, short sampling periods and applications that tolerate an occasional delay of one sampling period. Tree ordering is preferable

whenever latency is more important than the channel time it requires.

REFERENCES

- [1] C. Benrebbouh and L. Louail, "Dynamic TDMA for wireless sensor networks," in *Proc. 30th Wireless and Optical Communications Conference (WOCC)*, 2021, pp. 204–208.
- [2] X. Vilajosana, Q. Wang, F. Chraïm, T. Watteyne, T. Chang, and K. S. J. Pister, "A realistic energy consumption model for TSCH networks," *IEEE Sensors Journal*, vol. 14, no. 2, pp. 482–489, 2014.
- [3] S. C. Ergen and P. Varaiya, "TDMA scheduling algorithms for wireless sensor networks," *Wireless Networks*, vol. 16, no. 4, pp. 985–997, 2010.
- [4] W. Ye, J. Heidemann, and D. Estrin, "An energy-efficient MAC protocol for wireless sensor networks," in *Proc. IEEE INFOCOM*, vol. 3, 2002, pp. 1567–1576.
- [5] A. Bhatia and R. C. Hansdah, "A classification framework for TDMA scheduling techniques in WSNs," arXiv preprint arXiv:2002.00458, 2020.
- [6] A. R. Urke, Ø. Kure, and K. Øvsthus, "A survey of 802.15.4 TSCH schedulers for a standardized industrial Internet of Things," *Sensors*, vol. 22, no. 1, 2022, art. no. 15.
- [7] I. Rhee, A. Warriier, M. Aia, and J. Min, "Z-MAC: A hybrid MAC for wireless sensor networks," in *Proc. 3rd ACM Conf. Embedded Networked Sensor Systems (SenSys)*, 2005, pp. 90–101.
- [8] I. A. Bouchedjra and L. Louail, "Latency and energy efficient routing-aware TDMA for wireless sensor networks," *International Journal of Informatics and Applied Mathematics*, vol. 3, no. 1, pp. 22–38, 2020.
- [9] Ö. Durmaz Incel, A. Ghosh, B. Krishnamachari, and K. Chintalapudi, "Fast data collection in tree-based wireless sensor networks," *IEEE Transactions on Mobile Computing*, vol. 11, no. 1, pp. 86–99, 2012.
- [10] C. Benrebbouh, L. Louail, and S. Cherbal, "Distributed TDMA for IoT using a dynamic slot assignment," in *Proc. Int. Conf. Advances in Computing Research (ACR)*, ser. Lecture Notes in Networks and Systems. Springer, 2023, pp. 469–480.
- [11] B. M. Chittapragada, R. Kumar, V. Doley, and B. R. Chandavarkar, "Improving dynamic TDMA for wireless sensor networks," in *Proc. 14th Int. Conf. Computing Communication and Networking Technologies (ICCCNT)*, 2023, pp. 1–7.
- [12] V. Rajendran, K. Obraczka, and J. J. Garcia-Luna-Aceves, "Energy-efficient, collision-free medium access control for wireless sensor networks," *Wireless Networks*, vol. 12, no. 1, pp. 63–78, 2006.